

TomatoGuard AI

Backend Implementation Specification

FastAPI · JWT · Cloudinary · phase-by-phase, v1 scope

How to Use This Document

Companion to the Database document — read that one first, every table/column name here is exact and matches it. This document assumes the v1 scope decisions: one farm per farmer (auto-created at signup), Tasks/Schemes/Notifications/Reports cut entirely, Admin included as a parallel module, soil ML model treated as optional. Build phases in order; Phases 3, 4, 5 can be split across different developers/models once Phase 2 is done, since none of them depend on each other. The Admin Module has no dependency on Phases 2–6 either and can be built by a fourth person in parallel from day one.

Repo layout (matches your team's agreed convention)

```

TomatoGuard-AI/
├── frontend/
├── backend/
│   └── app/
│       ├── main.py
│       ├── core/          config.py, db.py, security.py, cloudinary_client.py
│       ├── models/        one SQLAlchemy model per table
│       ├── schemas/       one Pydantic schema file per domain
│       ├── api/           one router per domain
│       └── services/       disease_model.py, ocr.py, soil_analysis.py,
weather_client.py
├── database/
│   └── schema.sql         the migration script from the Database document
└── models/               trained model artifacts (.h5/.tflite) + README
  
```

Cloudinary integration (used by Phases 3 and 4)

- `app/core/cloudinary_client.py` — a single `upload_image(file_bytes, folder) -> (secure_url, public_id)` function. Configure via `CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET` env vars, never hardcoded.
- The frontend NEVER talks to Cloudinary directly — it always uploads to your FastAPI backend, which uploads to Cloudinary server-side. This keeps your API secret off the client, same principle as your team's own 'never connect frontend directly to Postgres' rule.
- Folder convention: `tomatoguard/{user_id}/disease_scans/` and `tomatoguard/{user_id}/soil_reports/` — keeps the media library navigable per farmer.
- Store both the returned `secure_url` (in `image_path` / `report_file_path`) and `public_id` (in the new `image_public_id` / `report_file_public_id` columns) — the `public_id` is what you'd need for a future delete/replace feature.

Phase 0 — Environment & Repo Setup

- `docker-compose.yml`: backend + frontend services. No local Postgres container needed — you're using Neon directly, so `DATABASE_URL` points at your Neon connection string even in local dev.
- `backend/requirements.txt`: `fastapi`, `uvicorn[standard]`, `sqlalchemy`, `psycopg2-binary`, `python-jose[cryptography]`, `passlib[bcrypt]`, `bcrypt==4.0.1` (pin explicitly — newer `bcrypt` breaks `passlib` 1.7.4), `python-multipart`, `pydantic-settings`, `cloudinary`, `pytesseract`, `Pillow`.
- backend Dockerfile installs the `tesseract-ocr` system package (`apt-get`) for Phase 4.

Acceptance criteria

- `docker-compose up` starts cleanly.
- Backend connects to the real Neon database on startup with no connection errors.
- `GET /health` returns `{"status":"ok"}`.

Phase 1 — Authentication + Farm Auto-Creation

Objective: JWT auth, exactly as before, PLUS the v1 rule that every signup automatically creates the farmer's one farms row — this is the key difference from a generic auth phase.

Backend — files to create

- `app/core/security.py` — `hash_password/verify_password` (bcrypt), `create_access_token/create_refresh_token/decode_token` (JWT), `get_current_user()` dependency. Unchanged from a standard implementation.
- `app/models/user.py`, `app/models/farm.py` — SQLAlchemy models matching the Database doc sections 2.1 and 2.2 exactly, including the UNIQUE constraint on `farms.user_id`.

POST `/auth/signup`

Create a farmer account AND their one farm row, in a single transaction.

Request body / params:

- name, email, password (required)
- phone, address, village, district, state (optional)
- farm_name (required — even a placeholder like "My Farm" if the farmer doesn't name it)
- latitude, longitude (optional — used by Phase 5)

Response fields:

- access_token, refresh_token, token_type

Server-side logic (implement in this exact order):

1. Check no existing user has this email — 409 if so.
2. Hash the password.
3. Insert the users row.
4. Insert exactly one farms row with that user_id, farm_name, latitude, longitude — both inserts happen in the same DB transaction; if either fails, roll back both.
5. Issue and return tokens.

POST `/auth/login`

Authenticate.

Request body / params:

- email, password

Response fields:

- access_token, refresh_token, token_type

Server-side logic (implement in this exact order):

1. Look up by email.
2. Verify password hash — 401 with a generic message if either the email or password is wrong.
3. If `is_active` is false, return 403 with a message like "This account has been deactivated" — do not use the generic 401 here, since this is a distinct, admin-driven state the farmer should understand.
4. Issue tokens with `aud:'farmer'`.

POST `/auth/refresh`

Rotate tokens.

Request body / params:

- refresh_token

Response fields:

- access_token, refresh_token, token_type

Server-side logic (implement in this exact order):

1. Decode, verify type='refresh'.
2. Issue a new access + refresh pair.

GET /auth/me

Current user + their farm — the frontend needs both after a page refresh.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- user_id, name, email, village/district/state, farm_id, farm_name, latitude, longitude

Server-side logic (implement in this exact order):

1. Decode token, look up user.
2. Join to their single farms row (guaranteed to exist — created at signup).
3. Return combined object; frontend never has to make a second call to learn its own farm_id.

Acceptance criteria

- Signup creates exactly one users row and exactly one farms row, correctly linked.
- If farm creation fails for any reason, the user row is also rolled back — never a user with no farm.
- GET /auth/me returns farm_id without a second API call.

Admin Module (build in parallel with Phase 2 onward — no dependency between them)

A separate, parallel authentication track from farmer auth. Admin tokens and farmer tokens must never be interchangeable — this is enforced via a token 'aud' (audience) claim, checked by two entirely separate FastAPI dependencies.

Backend — files to create

- `app/models/admin.py` — SQLAlchemy model matching the Database doc section 2.0.
- `app/core/security.py` — extend with `create_admin_access_token()/get_current_admin()` alongside the existing farmer versions. Both use the same JWT secret and algorithm, but encode `aud:'admin'` vs `aud:'farmer'` in the payload — `get_current_admin()` rejects any token where `aud != 'admin'`, and `get_current_user()` rejects any token where `aud != 'farmer'`. This is the single most important line of code in this module — a bug here means a farmer token could access admin routes.
- `app/api/admin_auth.py`, `app/api/admin_diseases.py`, `app/api/admin_farmers.py` — routers below.

POST `/admin/login`

Authenticate an admin. No signup endpoint exists — accounts are created directly in the database (see Database doc, section 4B).

Request body / params:

- email, password

Response fields:

- `access_token`, `refresh_token`, `token_type`

Server-side logic (implement in this exact order):

1. Look up by email in the admin table (never the users table).
2. Verify password hash — 401 with a generic message if wrong.
3. Issue tokens with `aud:'admin'`.

Diseases management

GET `/admin/diseases`

List all disease lookup entries.

Request body / params:

- Authorization: Bearer <admin access_token>

Response fields:

- array of full diseases rows

Server-side logic (implement in this exact order):

1. `get_current_admin()` dependency.
2. Return all rows from diseases.

POST `/admin/diseases`

Add a new disease entry.

Request body / params:

- disease_name, description, symptoms, causes, prevention, treatment

Response fields:

- the created row

Server-side logic (implement in this exact order):

1. get_current_admin() dependency.
2. Insert into diseases — disease_name must be unique (409 if it collides with an existing one, e.g. a farmer-facing scan already used that exact label).

PATCH /admin/diseases/{disease_id}

Edit an existing disease entry.

Request body / params:

- any subset of the disease fields

Response fields:

- the updated row

Server-side logic (implement in this exact order):

1. get_current_admin() dependency.
2. Update only the fields provided; bump updated_at.

Farmer oversight

GET /admin/farmers

List all farmers with basic farm info.

Request body / params:

- Authorization: Bearer <admin access_token>
- Query: search (optional, matches name/email/village)

Response fields:

- array of {user_id, name, email, village, district, is_active, farm_name, created_at}

Server-side logic (implement in this exact order):

1. get_current_admin() dependency.
2. Join users to farms, apply the optional search filter, return paginated if the list grows (offset/limit query params — implement pagination now even if v1 has few farmers, since retrofitting it later touches the frontend list UI too).

GET /admin/farmers/{user_id}

Full detail on one farmer, including their recent activity.

Request body / params:

- Authorization: Bearer <admin access_token>

Response fields:

- farmer profile + farm + latest disease_scans (limit 5) + latest soil_analyses (limit 5) + latest weather_records (limit 5)

Server-side logic (implement in this exact order):

1. get_current_admin() dependency.

2. 404 if user_id doesn't exist.
3. This reuses the same queries as the farmer's own /dashboard/summary and /history, just scoped by admin permission instead of the token's own user_id.

PATCH /admin/farmers/{user_id}/status

Activate or deactivate a farmer account.

Request body / params:

- is_active (boolean)

Response fields:

- the updated user_id and is_active value

Server-side logic (implement in this exact order):

1. get_current_admin() dependency.
2. Update users.is_active.
3. A deactivated farmer's existing JWT access tokens remain valid until they naturally expire (max 30 min) since there's no token revocation list in v1 — acceptable at this scale, but worth knowing: deactivation isn't instant.

Farmer-side enforcement: get_current_user() (used by every farmer endpoint) must also check is_active and return 403 if false — otherwise deactivation only blocks new logins, not an already-issued token's continued use within its 30-minute window. Add this one check to the existing dependency rather than duplicating it per-route.

Frontend — v1 admin pages (new, not in the existing farmer-facing pages)

- src/pages/admin/AdminLogin.jsx — separate from farmer Login, posts to /admin/login, stores the token under a different localStorage key (e.g. tomatoAdminTokens) so an admin session and a farmer session can't collide if both are open in the same browser.
- src/pages/admin/AdminDashboard.jsx — farmer list (search + pagination), links into detail view.
- src/pages/admin/AdminFarmerDetail.jsx — one farmer's profile + recent activity + activate/deactivate toggle.
- src/pages/admin/AdminDiseases.jsx — table of disease entries, add/edit form.
- A separate AdminProtected route wrapper checking tomatoAdminTokens, mounted at /admin/* — entirely separate from the farmer Protected wrapper.

Acceptance criteria

- A farmer's JWT cannot access any /admin/* route (403), and an admin's JWT cannot access any farmer route (403) — test both directions explicitly.
- Admin can add a disease entry and it immediately appears correctly matched on the next disease scan with that label.
- Deactivating a farmer sets is_active=false; that farmer's next login attempt fails even with correct credentials.
- Admin farmer list and detail views show real data only — no fabricated placeholder rows.

Phase 2 — Dashboard Shell & PWA

GET /dashboard/summary

Populates the four dashboard summary cards.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- latest_disease_scan (or null), latest_soil_analysis (or null), latest_weather (or null)

Server-side logic (implement in this exact order):

1. Get the user's farm_id from the token (via the same lookup as /auth/me).
2. Query the most recent row from disease_scans, soil_analyses, and weather_records filtered by that farm_id.
3. Return null for any that don't exist — never fabricate placeholder values.

PWA setup (manifest, service worker, icons, camera-first capture attribute) is unchanged from a standard React+Vite PWA build — see your team's own PDF, section 2, which already lists this as the correct stack (vite-plugin-pwa equivalent tooling).

Acceptance criteria

- Dashboard loads with all cards in explicit empty states on a brand-new account.
- App is installable (Lighthouse PWA audit green).

Phase 3 — Disease Detection

- `app/services/disease_model.py` — `DiseaseModel` class, `predict(image_bytes) -> (label, confidence)`. Load once at import time.
- Model hosting: if the `.h5/.tflite` file is small enough for your hosting tier's memory budget, load in-process; otherwise convert to TFLite first (~4x smaller memory footprint, same accuracy).

POST `/disease/analyze`

Upload a leaf photo, run inference, store the result.

Request body / params:

- `multipart/form-data`: image (file)
- Authorization: Bearer <access_token>

Response fields:

- `scan_id`, `predicted_disease`, `confidence`, `description`, `immediate_action`

Server-side logic (implement in this exact order):

1. Validate image type/size (reject >8MB or non-image with 400).
2. Upload the image to Cloudinary via `upload_image()` — get back `secure_url` + `public_id`.
3. Run `DiseaseModel.predict()` on the image bytes.
4. Look up `disease_id` from the `diseases` table by matching `disease_name` = predicted label (null if no match).
5. Insert into `disease_scans`: `user_id`, `farm_id` (from the user's one farm), `disease_id`, `image_path`=`secure_url`, `image_public_id`, `predicted_disease`, `confidence`, `model_name`='EfficientNetB0'.
6. If `disease_id` matched, pull `description/treatment` from the `diseases` row for the response; otherwise return the raw label only.

GET `/disease/history`

This farmer's past scans.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- array of {`scan_id`, `predicted_disease`, `confidence`, `scan_date`}

Server-side logic (implement in this exact order):

1. Query `disease_scans` filtered by `user_id`, ordered `scan_date` DESC.

Acceptance criteria

- A real leaf photo produces a real label + confidence.
- The image is retrievable via its stored Cloudinary URL.
- An oversized/non-image upload returns 400, not 500.

Phase 4 — Soil Health

Two independent input paths (report upload with OCR, or questionnaire) converging on one `soil_analyses` row. Server-side OCR only — this was a deliberate fix from an earlier client-side `tesseract.js` implementation, which conflicts with the app's low-bandwidth mobile-first requirement.

- `app/services/ocr.py` — `run_ocr(image_bytes) -> str (pytesseract) + parse_soil_fields(text) -> dict` (regex extraction of N/P/K/pH/EC/organic matter).
- `app/services/soil_analysis.py` — `a generate_analysis(ph, n, p, k, moisture, organic_matter) -> dict` rule engine producing `fertilizer_recommendation`, `irrigation_recommendation`, and a fertility rating from whichever fields are non-null.

Soil ML model — unconfirmed: `predicted_soil_condition`, `confidence`, `model_name`, `model_version` stay NULL unless your team confirms and delivers a trained soil-classification model. Build `soil_analysis.py` as a rule-based engine now; if a real model arrives later, it's a drop-in replacement for that one function — the endpoint contract below doesn't change either way.

POST /soil/report

Upload a soil-test report image; OCR it and return extracted fields for review (not yet saved).

Request body / params:

- `multipart/form-data`: file (image)
- Authorization: Bearer <access_token>

Response fields:

- `soil_report_id`, `raw_ocr_text`, `fields`: {`ph`, `n`, `p`, `k`, `ec`, `organic_matter`} (each nullable)

Server-side logic (implement in this exact order):

1. Upload the image to Cloudinary.
2. Insert a `soil_reports` row (`report_file_path`, `report_file_public_id`) — save immediately, unlike the analysis itself.
3. Run OCR on the image, then `parse_soil_fields()`.
4. Return the `soil_report_id` plus raw text and parsed fields — the frontend shows these as editable before the farmer confirms.

POST /soil/report/confirm

Persist the (possibly corrected) OCR fields as an analysis.

Request body / params:

- `soil_report_id` (from the previous call)
- `ph`, `n`, `p`, `k`, `ec`, `organic_matter` (farmer-reviewed values)
- Authorization: Bearer <access_token>

Response fields:

- `analysis_id`, `fertility_rating`, `fertilizer_recommendation`, `irrigation_recommendation`

Server-side logic (implement in this exact order):

1. Call `generate_analysis()` with the confirmed values.
2. Insert a `soil_analyses` row with `soil_report_id` set, `questionnaire_id` null.
3. Return the generated advice.

POST /soil/questionnaire

Submit the manual Q&A path.

Request body / params:

- crop_stage, previous_crop, irrigation_type, fertilizer_used, soil_color, drainage_condition (strings)
- Authorization: Bearer <access_token>

Response fields:

- analysis_id, fertility_rating, fertilizer_recommendation, irrigation_recommendation

Server-side logic (implement in this exact order):

1. Insert a soil_questionnaires row.
2. Call generate_analysis() — for this path, ph/n/p/k/moisture/organic_matter all stay null; the rule engine must handle an all-null nutrient case gracefully (fall back to qualitative rules based on soil_color/drainage_condition alone).
3. Insert a soil_analyses row with questionnaire_id set, soil_report_id null.

GET /soil/latest

Most recent soil analysis for this farmer.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- soil_analyses fields, or null

Server-side logic (implement in this exact order):

1. Query soil_analyses filtered by user_id, most recent by analysis_date.

Acceptance criteria

- Report upload → review → confirm creates exactly one soil_analyses row linked to the soil_report_id.
- Questionnaire path creates exactly one soil_analyses row linked to the questionnaire_id, with null nutrient fields handled gracefully by the rule engine.
- GET /soil/latest reflects whichever path was submitted most recently.

Phase 5 — Weather

GET /weather/current

Fetch and log current conditions for the farmer's farm location.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- temperature_c, humidity_percent, rainfall_mm, wind_speed_kmh, weather_condition

Server-side logic (implement in this exact order):

1. Read latitude/longitude from the user's farm — 400 with a clear message if both are null.
2. Call OpenWeatherMap's current-weather endpoint.
3. Insert a weather_records row (this is also what History/Dashboard read from).
4. Return the mapped fields.

GET /weather/forecast

3-5 day forecast.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- array of {date, temperature_c, condition}

Server-side logic (implement in this exact order):

1. Same location precondition.
2. Call the forecast endpoint — do NOT write to weather_records for forecast calls, only for /current, to keep the history table meaningful.

Acceptance criteria

- Real current weather + forecast for a farm with a saved location.
- Clear prompt (not an error page) for a farm with no location set.
- Each /weather/current call adds exactly one weather_records row.

Phase 6 — Recommendations (computed, not persisted)

Your real schema has no recommendations table. Rather than add one (touching your teammate's already-deployed schema for a v1-only need), this phase computes the combined view on request from the three domains' latest rows.

GET /recommendations/latest

Combine the farmer's latest disease scan, soil analysis, and weather record into one banner.

Request body / params:

- Authorization: Bearer <access_token>

Response fields:

- health_status ('good'/'at-risk'/'critical'), disease_summary, soil_summary, weather_summary, fertilizer_advice, irrigation_advice, treatment_advice

Server-side logic (implement in this exact order):

1. Fetch the farmer's latest disease_scans, soil_analyses, and weather_records rows (any may be null).
2. If all three are missing, return 404 — nothing to show yet.
3. Run a small rules function combining disease severity + soil fertility_rating + weather condition into an overall health_status.
4. Pull fertilizer_recommendation/irrigation_recommendation straight from the soil_analyses row, and recommendation/treatment from disease_scans/diseases — this endpoint mostly assembles existing data, it doesn't invent new advice.
5. This is computed fresh on every call — there's no caching or persistence to keep in sync, which is the main advantage of not having a dedicated table for v1.

Acceptance criteria

- With real disease + soil + weather data present, the endpoint returns a coherent combined banner.
- With zero data, returns 404, not an empty/fake success.
- With partial data (e.g. disease + weather but no soil), returns a partial result — this endpoint must not require all three, only combine whatever exists.